

Scanned Code Report

AUDITAGENT

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Auditor Scan

#	Scan ID	+	Date
	9		July 23, 2026
	Organization		Repository
	CMTA		CMTAT
	Branch		Commit Hash
	master		35d8940b...9d92e4ae

Contracts in scope

contracts/deployment/CMTATStandardStandalone.sol

contracts/deployment/CMTATStandardUpgradeable.sol

contracts/deployment/CMTATUpgradeableUUPS.sol

contracts/deployment/ERC1363/CMTATStandaloneERC1363.sol

contracts/deployment/ERC1363/CMTATUpgradeableERC1363.sol

contracts/deployment/ERC7551/CMTATStandaloneERC7551.sol

contracts/deployment/ERC7551/CMTATUpgradeableERC7551.sol

contracts/deployment/allowlist/CMTATStandaloneAllowlist.sol

contracts/deployment/allowlist/CMTATUpgradeableAllowlist.sol

contracts/deployment/debt/CMTATStandaloneDebt.sol

contracts/deployment/debt/CMTATUpgradeableDebt.sol

contracts/deployment/debtEngine/CMTATStandaloneDebtEngine.sol

contracts/deployment/debtEngine/CMTATUpgradeableDebtEngine.sol

contracts/deployment/holderList/CMTATStandaloneHolderList.sol

contracts/deployment/holderList/CMTATUpgradeableHolderList.sol

contracts/deployment/permit/CMTATStandalonePermit.sol

contracts/deployment/permit/CMTATUpgradeablePermit.sol

contracts/deployment/snapshot/CMTATStandaloneSnapshot.sol

contracts/deployment/snapshot/CMTATUpgradeableSnapshot.sol

contracts/interfaces/engine/IDebtEngine.sol contracts/interfaces/engine/IDocumentEngine.sol

contracts/interfaces/engine/IRuleEngine.sol contracts/interfaces/engine/ISnapshotEngine.sol

contracts/interfaces/modules/IAllowlistModule.sol contracts/interfaces/modules/IDebtEngineModule.sol

contracts/interfaces/modules/IDebtModule.sol contracts/interfaces/modules/IDocumentEngineModule.sol

contracts/interfaces/modules/IHolderListModule.sol

contracts/interfaces/modules/ISnapshotEngineModule.sol

contracts/interfaces/technical/ICMTATConstructor.sol contracts/interfaces/technical/IERC20Allowance.sol

contracts/interfaces/technical/IERC5679.sol contracts/interfaces/technical/IERC7802.sol

contracts/interfaces/technical/IGetCCIPAdmin.sol contracts/interfaces/technical/IMintBurnToken.sol

contracts/interfaces/tokenization/ICMTAT.sol contracts/interfaces/tokenization/IERC3643Partial.sol

contracts/interfaces/tokenization/draft-IERC1404.sol contracts/interfaces/tokenization/draft-IERC1643.sol

contracts/interfaces/tokenization/draft-IERC1643CMTAT.sol

contracts/interfaces/tokenization/draft-IERC7551.sol contracts/interfaces/tokenization/draft-IERC7943.sol

contracts/interfaces/tokenization/draft-IERC8343.sol contracts/library/ERC1404ExtendInterfaceId.sol

contracts/library/RuleEngineInterfaceId.sol contracts/modules/0_CMTATBaseCommon.sol

contracts/modules/0_CMTATBaseCore.sol contracts/modules/0_CMTATBaseGeneric.sol

contracts/modules/0_CMTATBaseSnapshot.sol contracts/modules/1_CMTATBaseDocument.sol

contracts/modules/2_CMTATBaseAccessControl.sol contracts/modules/3_CMTATBaseAllowlist.sol

contracts/modules/3_CMTATBaseRuleEngine.sol contracts/modules/4_CMTATBaseDebt.sol

contracts/modules/4_CMTATBaseERC1404.sol contracts/modules/5_CMTATBaseERC20CrossChain.sol

contracts/modules/6_CMTATBaseDebtEngine.sol contracts/modules/6_CMTATBaseERC2612.sol

contracts/modules/6_CMTATBaseERC2771.sol contracts/modules/7_CMTATBaseERC2771Snapshot.sol

contracts/modules/7_CMTATBaseERC7551Enforcement.sol

contracts/modules/8_CMTATBaseERC1363.sol contracts/modules/8_CMTATBaseERC7551.sol

contracts/modules/8_CMTATBaseHolderList.sol contracts/modules/internal/AllowlistModuleInternal.sol

contracts/modules/internal/ERC20BurnModuleInternal.sol

contracts/modules/internal/ERC20EnforcementModuleInternal.sol

contracts/modules/internal/ERC20MintModuleInternal.sol

contracts/modules/internal/EnforcementModuleInternal.sol

contracts/modules/internal/ValidationModuleRuleEngineInternal.sol

contracts/modules/internal/common/EnforcementModuleLibrary.sol

contracts/modules/wrapper/controllers/ValidationModule.sol

contracts/modules/wrapper/controllers/ValidationModuleAllowlist.sol

contracts/modules/wrapper/core/ERC20BaseModule.sol

contracts/modules/wrapper/core/ERC20BurnModule.sol

contracts/modules/wrapper/core/ERC20MintModule.sol

contracts/modules/wrapper/core/EnforcementModule.sol contracts/modules/wrapper/core/PauseModule.sol

contracts/modules/wrapper/core/TokenAttributeModule.sol

contracts/modules/wrapper/core/ValidationModuleCore.sol

contracts/modules/wrapper/core/VersionModule.sol

contracts/modules/wrapper/extensions/DocumentERC1643Module.sol

```
contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol
contracts/modules/wrapper/extensions/ExtraInformationModule.sol
contracts/modules/wrapper/extensions/SnapshotEngineModule.sol
contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleAllowance.sol
contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleERC1404.sol
contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol
contracts/modules/wrapper/options/AllowlistModule.sol contracts/modules/wrapper/options/CCIPModule.sol
contracts/modules/wrapper/options/DebtEngineModule.sol
contracts/modules/wrapper/options/DebtModule.sol
contracts/modules/wrapper/options/DocumentEngineModule.sol
contracts/modules/wrapper/options/ERC20CrossChainModule.sol
contracts/modules/wrapper/options/ERC20EnforcementERC7551Module.sol
contracts/modules/wrapper/options/ERC2771Module.sol
contracts/modules/wrapper/options/ERC7551Module.sol
contracts/modules/wrapper/options/HolderListModule.sol
contracts/modules/wrapper/security/AccessControlModule.sol
contracts/deployment/light/CMTATUpgradeableLight.sol
```

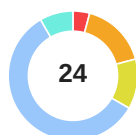
Code Statistics

 Findings
24

 Contracts Scanned
100

 Lines of Code
8067

Findings Summary



Total Findings

 High Risk (1)
 Medium Risk (4)
 Low Risk (3)

 Info (14)
 Best Practices (2)

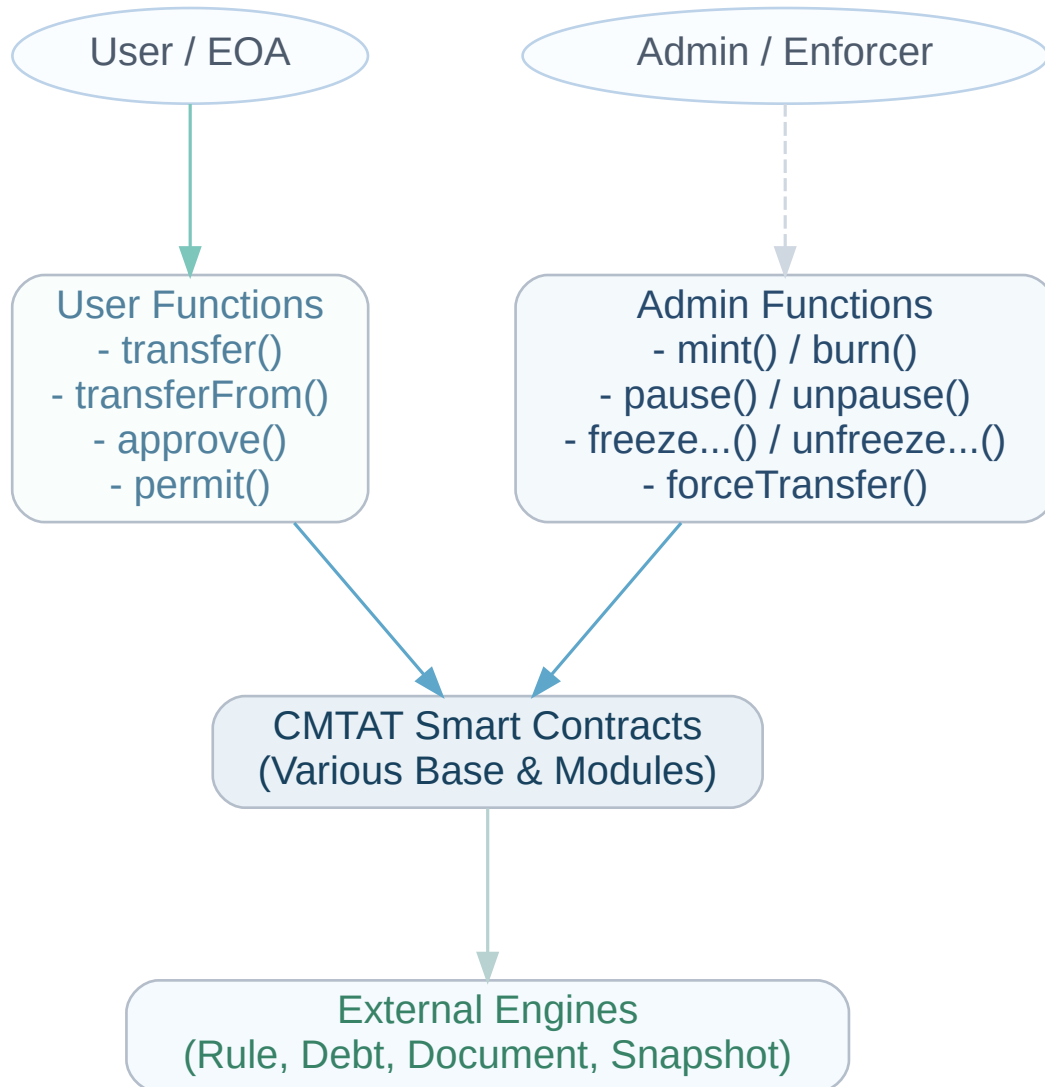
Code Summary

The protocol provides a modular framework for deploying and managing tokenized assets compliant with the Capital Markets Technology Association (CMTAT) specifications and Swiss law. It includes various smart contracts designed to support standalone or proxy-based deployments with a highly flexible architecture. The protocol is structured around a core ERC-20 token standard, enhanced with functionalities for compliance (ERC-1404, ERC-3643, ERC-7551), gasless transactions (ERC-2771), cross-chain operations (ERC-7802, Chainlink CCIP), document management (ERC-1643), debt instruments, snapshot engines, on-chain holder tracking, and allowlisting. The modular design enables issuers to mix and match functionalities tailored to the specific regulatory and operational requirements of tokenized financial instruments and real-world assets (RWAs).

Main entry points and actors:

- `initialize`: Called by the deployer to initialize the proxy implementation with core attributes like admin address, token details, terms, and external engines.
- `transfer` / `transferFrom`: Standard ERC-20 transfers used by token holders and authorized spenders (subject to compliance rules).
- `approve`: Used by token holders to grant spenders the right to transfer tokens.
- `burn` / `batchBurn`: Called by actors with the `BURNER_ROLE` (or `BURNER_SELF_ROLE` / `BURNER_FROM_ROLE` for cross-chain specific roles) to destroy tokens.
- `mint` / `batchMint` / `batchTransfer`: Called by actors with the `MINTER_ROLE` to create new tokens and distribute them.
- `burnAndMint`: Allows burning and minting tokens atomically, restricted to callers holding required roles.
- `forcedBurn` / `forcedTransfer`: Called by compliance or admin actors (`DEFAULT_ADMIN_ROLE` or `ERC20ENFORCER_ROLE`) to unilaterally burn or transfer tokens, typically from frozen accounts for regulatory reasons.
- `setAddressFrozen` / `batchSetAddressFrozen`: Called by actors with the `ENFORCER_ROLE` to freeze or unfreeze accounts.
- `freezePartialTokens` / `unfreezePartialTokens` / `setFrozenTokens`: Called by the `ERC20ENFORCER_ROLE` to restrict specific amounts of a token holder's balance.
- `pause` / `unpause`: Called by actors with the `PAUSER_ROLE` to halt or resume token transfers.
- `deactivateContract`: Called by the `DEFAULT_ADMIN_ROLE` to permanently deactivate the token contract.
- `setDocument` / `removeDocument`: Called by the `DOCUMENT_ROLE` to manage off-chain document references linked to the token.
- `setTokenId` / `setTerms` / `setInformation` / `setMetaData`: Called by the `EXTRA_INFORMATION_ROLE` to update token metadata and terms.
- `setName` / `setSymbol`: Called by the `DEFAULT_ADMIN_ROLE` to update basic token attributes.
- `setRuleEngine` / `setDebtEngine` / `setSnapshotEngine` / `setDocumentEngine`: Called by specific admin roles (`DEFAULT_ADMIN_ROLE`, `DEBT_ENGINE_ROLE`, `SNAPSHOTTER_ROLE`, `DOCUMENT_ENGINE_ROLE`) to update external engine integrations.
- `setDebt` / `setDebtInstrument` / `setCreditEvents`: Called by the `DEBT_ROLE` to manage on-chain debt token attributes.
- `setAddressAllowlist` / `batchSetAddressAllowlist` / `enableAllowlist`: Called by the `ALLOWLIST_ROLE` to manage the list of authorized addresses.
- `permit`: Used by token holders (via a relay) to approve spenders using EIP-712 signatures.
- `crosschainMint` / `crosschainBurn`: Called by the `CROSS_CHAIN_ROLE` (trusted bridge) to mint or burn tokens across chains.
- `setCCIPAdmin`: Called by the `DEFAULT_ADMIN_ROLE` to update the Chainlink CCIP administrator address.

Code Diagram



1 of 24
Findings

contracts/modules/3_CMTATBaseAllowlist.sol

contracts/deployment/allowlist/CMTATUpgradeableAllowlist.sol

Uninitialized proxy can be seized by the first caller to `initialize`

• High Risk

`initialize(...)` is `public` and protected only by OpenZeppelin's `initializer` modifier. On a fresh proxy instance, that means any account can successfully execute the first initialization call as long as the proxy storage has not been initialized yet.

In the successful path, `initialize(...)` forwards into `__CMTAT_init(...)`, which reaches `__CMTAT_commonModules_init_unchained(...)`, and that in turn calls `__AccessControlModule_init_unchained(admin)` with the caller-supplied `admin` value. This grants `DEFAULT_ADMIN_ROLE` to the attacker-controlled address. Because `AccessControlModule.hasRole(...)` treats `DEFAULT_ADMIN_ROLE` as implicitly holding every role, the attacker immediately gains effective control over all privileged token operations.

For an allowlist deployment behind a proxy, this lets the attacker take over the token before the intended operator initializes it: they can mint arbitrary supply, freeze or unfreeze accounts, manage the allowlist, pause or deactivate the token, and execute forced-transfer style recovery paths. In practice, any proxy deployed without atomic initialization can be permanently captured by the first external caller, leading to full administrative takeover and potential theft or permanent denial of service.

Severity Note:

- The deployment process can deploy a Transparent or Beacon proxy without constructor/deployment calldata that atomically invokes `initialize`; this is not demonstrated by the supplied deployment scripts.
- The inherited modules expose the described privileged minting and/or token-enforcement capabilities, or token integrations will treat the attacker-controlled initialized token as the intended asset. The supplied excerpt directly establishes control over pause, deactivation, freeze, allowlist, and enforcement authorization, but does not include the external mint implementation.
- A separately controlled proxy admin does not prevent the initial public call to the proxy's implementation `initialize` function.

✦ 2 of 24 Findings

contracts/modules/0_CMTATBaseCore.sol

Anyone can claim an uninitialized proxy by calling initialize first

• Medium Risk

`initialize(...)` is a public entrypoint protected only by OpenZeppelin's one-time `initializer` latch. On a proxy instance whose storage has not been initialized yet, there is no caller authorization at all before the initial admin is assigned.

```
function initialize(  
    address admin,  
    ICMATConstructor.ERC20Attributes memory ERC20Attributes_  
) public virtual initializer {  
    __CMTAT_init(admin, ERC20Attributes_);  
}
```

During initialization, `__AccessControlModule_init_unchained(admin)` grants `DEFAULT_ADMIN_ROLE` to the supplied `admin` address. Because this project overrides `hasRole` so that a default admin is treated as having every role, the first successful caller can install an attacker-controlled admin and immediately obtain effective control over privileged token operations such as minting, burning, pausing, freezing, metadata changes, and role administration. The initializer is then consumed, so the legitimate operator cannot recover ownership by calling `initialize` afterward.

This is a full-token takeover on any proxy deployment that is created without atomically executing the initializer in the same transaction. The implementation contract itself is protected by `_disableInitializers()` in upgradeable deployment wrappers, but the proxy remains claimable until its own storage is initialized.

Severity Note:

- This assessment assumes deployments can actually leave a proxy uninitialized and externally callable, rather than a factory always deploying and initializing atomically.
- The supplied excerpt does not include `AccessControlModule`; the conclusion that the attacker can grant or effectively exercise `MINTER_ROLE/BURNER_ROLE`/other roles relies on the finding's stated default-admin role semantics and/or standard `AccessControl` role administration.

contracts/modules/3_CMTATBaseAllowlist.sol

3 of 24 Findings

contracts/modules/3_CMTATBaseRuleEngine.sol

contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleAllowance.sol

Paused or restricted holders cannot revoke stale allowances before transfers reactivate

• Medium Risk

Both exposed `approve` implementations apply the same value-independent authorization checks before every allowance update:

```
function approve(address spender, uint256 value)
    public
    virtual
    whenNotPaused
    returns (bool)
{
    _canAuthorizeAllowanceByModuleAndRevert(_msgSender(), spender);
    return ERC20Upgradeable.approve(spender, value);
}

function _canAuthorizeAllowanceByModuleAndRevert(
    address owner,
    address spender
) internal view virtual {
    _requireNotPaused();
    if (!_canSend(owner)) {
        revert ERC7943CannotSend(owner);
    }
    if (!_canSend(spender)) {
        revert ERC7943CannotSend(spender);
    }
}
```

The authorization hook does not receive `value` or inspect the current allowance. Consequently, an allowance revocation such as `approve(spender, 0)` is rejected under the same conditions as a new allowance grant: while the token is paused, when the owner or spender is fully frozen, or—in the allowlist variant—when either party is not allowlisted. Existing allowances are not cleared by any of those state transitions.

This creates a period where `transferFrom` is safely blocked but the owner is also unable to invalidate a previously granted allowance. As soon as the token is unpaused or the relevant account is unfrozen or re-allowlisted, the stale allowance becomes usable immediately. A compromised approved spender can monitor or back-run the restoration transaction and execute `transferFrom` before the owner's first valid revocation transaction. The amount exposed is bounded only by the remaining allowance and the owner's active balance. This also removes the safe opportunity to apply the zero-first allowance mitigation documented by the contracts while transfers are disabled.

Severity Note:

- CMTATBaseCommon.transferFrom ultimately performs the standard ERC-20 allowance consumption and token transfer after its validation checks, and transfer eligibility can in fact be restored for the owner, spender, and attacker-selected recipient.

- A completed unauthorized transfer is not automatically and reliably reversed by an external enforcement or forced-transfer process.

4 of 24
Findings

contracts/modules/5_CMTATBaseERC20CrossChain.sol

contracts/modules/6_CMTATBaseERC2771.sol

Inconsistent context resolution allows role spoofing and configuration-dependent unlimited minting

• Medium Risk

The protocol implements ERC-2771 meta-transactions via a trusted forwarder that overrides `_msgSender()` to authenticate users gaslessly. However, there is a critical inconsistency in how authorization is enforced across different privileged roles.

First, the bridge authentication intentionally relies on the raw `msg.sender`. In `3_CMTATBaseERC20CrossChain.sol`, the `onlyTokenBridge` modifier calls `_checkTokenBridge(msg.sender)`, which verifies if the direct caller has the `CROSS_CHAIN_ROLE`. Because of this, if a legitimate token bridge attempts to relay its calls through the trusted forwarder (e.g., using EIP-1271 to pay for gas), the transaction will revert since the forwarder itself lacks the `CROSS_CHAIN_ROLE`. If an administrator attempts to fix this broken integration by granting the `CROSS_CHAIN_ROLE` to the trusted forwarder, they create a catastrophic vulnerability: any user can then submit a meta-transaction to `crosschainMint`. The `onlyTokenBridge` check will pass (because `msg.sender` is the forwarder), and the function will mint arbitrary amounts of tokens to the user, completely bypassing the bridge authorization because `_msgSender()` is never checked against the role.

Second, unlike the bridge functions, all other highly privileged administrative roles (such as `DEFAULT_ADMIN_ROLE`, `MINTER_ROLE`, `PAUSER_ROLE`, and `ENFORCER_ROLE`) rely on the standard `onlyRole` modifier, which checks `_msgSender()`. This extends the trusted forwarder's authority far beyond standard user flows. If the forwarder contract is compromised or contains a signature spoofing vulnerability, an attacker can append the address of any admin to their calldata. The `_msgSender()` will resolve to the spoofed admin, allowing the attacker to impersonate role holders and completely take over the token contract.

Severity Note:

- The configured trusted forwarder accepts relayed calls from ordinary users after it has been granted `CROSS_CHAIN_ROLE`.
- `crosschainMint` ultimately mints the requested amount without an independent bridge-message proof or supply cap not shown in the provided code.
- An administrator may grant `CROSS_CHAIN_ROLE` to the forwarder to address the raw-`msg.sender` incompatibility.

5 of 24 Findings  contracts/modules/wrapper/controllers/ValidationModule.sol
contracts/modules/0_CMTATBaseCore.sol contracts/modules/0_CMTATBaseCommon.sol

Missing freeze enforcement on `spender` allows frozen accounts to execute `burnFrom` and minter transfers

• Medium Risk

The protocol enforces regulatory rules such as address freezing via central gates

(`ValidationModule._canTransferGenericByModuleAndRevert` and `CMTATBaseCommon._checkTransferred`).

However, the validation logic systematically fails to check the `spender`'s frozen status during burn operations and minter transfers.

In `ValidationModule._canTransferGenericByModuleAndRevert`, when a transaction is classified as a burn (`to == address(0)`), the validation routes to `_canBurnByModuleAndRevert(from)`. This function checks if the `from` address is frozen but completely ignores the `spender` (the caller). As a result, if an attacker's address is frozen by the protocol, they can still successfully invoke `burnFrom` to destroy tokens of any user who previously granted them an allowance.

Additionally, `CMTATBaseCore._minterTransferOverride` hardcodes `address(0)` as the `spender` instead of passing `_msgSender()` when routing to the generic module check. Because `address(0)` is naturally never frozen, this bypasses the freeze check for the minter executing the transfer, allowing a compromised and subsequently frozen minter address to continue moving user tokens via `minterTransfer`.

In `CMTATBaseCommon`, the `_checkTransferred` implementation similarly comments out and entirely ignores the `spender` argument across all operations.

contracts/modules/0_CMTATBaseCommon.sol

6 of 24 Findings

contracts/modules/3_CMTATBaseRuleEngine.sol

contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol

Zero-value delegated transfers can mutate RuleEngine state for arbitrary holders without any allowance

• Low Risk

`transferFrom` performs the validation hook before executing the ERC-20 allowance logic, and the RuleEngine-enabled override forwards every successful check to the external `ruleEngine.transferred(...)` callback even when no tokens actually move.

```
function transferFrom(address from, address to, uint256 value) public virtual returns (bool)
{
    _checkTransferred(_msgSender(), from, to, value);
    return ERC20BaseModule.transferFrom(from, to, value);
}
```

In RuleEngine deployments, `_checkTransferred` is overridden to call `_transferred(spender, from, to, value)`, which in turn calls `ruleEngine_.transferred(...)` for any `spender != address(0)`. Because OpenZeppelin's `transferFrom` treats `value == 0` as needing zero allowance, any caller can execute `transferFrom(victim, victim, 0)` or `transferFrom(victim, someAllowedReceiver, 0)` without prior approval from `victim`. The call succeeds, emits a zero-value `Transfer`, and still invokes the stateful RuleEngine callback for the victim tuple.

This lets an attacker manipulate compliance state for arbitrary holders without controlling their funds. If the configured RuleEngine updates cooldowns, holding periods, quotas, tax buckets, or similar accounting inside `transferred(...)` (which the interface explicitly permits), the attacker can repeatedly desynchronize policy state from balances or keep a victim in a permanently restricted state. In regulated-token deployments, that can amount to an effective freeze of the victim's assets even though no allowance was ever granted and no tokens were actually transferred.

Severity Note:

- The deployed RuleEngine's transferred callback mutates holder-relevant state even when value is zero.
- That state is used to impose a meaningful restriction or adverse accounting outcome, such as a cooldown or quota exhaustion.
- The RuleEngine does not independently reject zero-value events, self-transfers, or callbacks from an unapproved spender.
- The affected state cannot be immediately and automatically corrected without administrator or RuleEngine-operator action.

7 of 24
Findings

contracts/modules/8_CMTATBaseERC1363.sol

contracts/modules/3_CMTATBaseRuleEngine.sol

contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol

contracts/modules/0_CMTATBaseCommon.sol

Reentrant ruleEngine callback can bypass the active-balance freeze check and move frozen tokens

• Low Risk

`contracts/modules/8_CMTATBaseERC1363.sol::transfer()` is only a wrapper, but it routes into a transfer flow that performs an external `ruleEngine_.transferred(...)` callback before the sender's balance is updated.

The relevant order is:

1. `CMTATBaseCommon.transfer()` checks the sender's active balance through `_checkTransferred(...)`.
2. `CMTATBaseRuleEngine._checkTransferred()` calls `ValidationModuleRuleEngine._transferred(...)`.
3. `_transferred(...)` performs an external call to the configured RuleEngine.
4. Only after that call returns does `ERC20Upgradeable._transfer(...)` actually debit the sender.

Because the active-balance check runs before the external callback, a malicious or compromised RuleEngine can reenter `transferFrom(from, attacker, amount)` during `transferred(...)`. The reentrant call observes the same pre-transfer balance/frozen state as the outer call, so both calls can independently pass `_checkActiveBalanceAndRevert(...)` against the same active balance.

That breaks the intended freeze invariant. For example, if `balanceOf(from)=100` and `frozenTokens(from)=90`, a user transfer of `10` passes because the active balance is `10`. During the RuleEngine callback, a reentrant `transferFrom(from, attacker, 10)` also passes against the same `100/90` snapshot. When control returns, the outer `transfer` still succeeds because the final `ERC20Upgradeable._transfer` only checks total balance, not active balance. The result is that `20` tokens move even though only `10` were unfrozen, effectively consuming frozen funds.

The ERC-1363 wrapper preserves this vulnerable ordering unchanged, so calling `transfer()` through `CMTATBaseERC1363` exposes the same frozen-balance bypass.

8 of 24
Findings

contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleAllowance.sol

contracts/modules/0_CMTATBaseCore.sol contracts/modules/3_CMTATBaseAllowlist.sol

Allowance revocation is blocked for frozen or non-allowlisted spenders

• Low Risk

The `_canAuthorizeAllowanceByModuleAndRevert` guard validates that an allowance can be established between an `owner` and a `spender` by checking their eligibility (`_canSend`), which enforces that neither party is frozen or absent from the active allowlist. This guard is applied consistently on every `approve` override.

However, the guard does not evaluate the `value` of the allowance being set, applying these eligibility checks indiscriminately even when an owner is attempting to revoke an allowance (`value == 0`).

If a user approves a spender, and that spender is subsequently compromised, frozen, or removed from the allowlist by the protocol administrators, the user will be unable to revoke the spender's allowance. Any call to `approve(spender, 0)` will revert with `ERC7943CannotSend(spender)` because `_canSend(spender)` will evaluate to false.

This violates a critical risk-management invariant: users must always be able to sever ties with untrusted, sanctioned, or delisted actors. By locking the allowance state when the spender falls out of compliance, the protocol leaves the user exposed. If the spender's status is ever restored (e.g., a temporary suspension is lifted or a new allowlist criteria is met), they will instantly regain access to the user's unrevoked allowance and can potentially drain funds, having denied the user any window to protect themselves.

Severity Note:

- The previously frozen or delisted spender will later be restored to an eligible status.
- The restored spender can satisfy all transferFrom validation checks and has a remaining allowance against an owner with an available transferable balance.
- The spender executes transferFrom before the owner learns of reinstatement and successfully revokes the allowance.

contracts/modules/0_CMTATBaseCommon.sol

9 of 24 Findings

contracts/modules/3_CMTATBaseRuleEngine.sol

contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol

Rule-engine callback is reentrant before balance/allowance effects, enabling recursive transferFrom drains past frozen-balance limits

[• Info](#)

In rule-engine deployments, the transfer validation path performs an external callback before any ERC20 balance or allowance mutation occurs.

`contracts/modules/0_CMTATBaseCommon.sol:72-78` calls `_checkTransferred(...)` before `ERC20Upgradeable._transfer(...)`. In `contracts/modules/3_CMTATBaseRuleEngine.sol`, that hook is overridden to call `ValidationModuleRuleEngine._transferred(...)`, and `contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol` then executes `ruleEngine_.transferred(...)` before the pending transfer spends allowance or updates balances.

Because this external call happens before effects, a malicious or compromised rule engine can recursively call `transferFrom(from, attacker, chunk)` during the callback. Every nested frame re-runs `_checkActiveBalanceAndRevert(from, chunk)` against the same pre-transfer balance/frozen snapshot, so each call can pass as long as `chunk` is no larger than the victim's initial active balance. After recursion stops, the stack unwinds and each pending frame then spends allowance and executes `_transfer`. `_transfer` only checks total balance at execution time, so once the first nested transfer reduces the victim's active balance to zero, later frames still continue debiting what should have remained frozen.

As a result, if the rule-engine address has spender allowance from a partially frozen holder, the rule engine can drain up to the holder's full balance in one transaction by recursively stacking many small `transferFrom` calls, even though each individual call was only supposed to be limited by the holder's unfrozen amount. This turns a spender approval to the rule engine into a full freeze-bypass and fund-drain primitive.

10 of 24
Findings

contracts/modules/wrapper/security/AccessControlModule.sol
contracts/deployment/CMTATUpgradeableUUPS.sol

Documented two-step default-admin protection is absent

• Info

The documented access-control model states that `DEFAULT_ADMIN_ROLE` is protected by OpenZeppelin's `AccessControlDefaultAdminRules`, including a single-admin constraint, delayed transfer, and explicit acceptance by the new administrator. The implementation instead directly inherits unrestricted `AccessControlUpgradeable`:

```
abstract contract AccessControlModule is AccessControlUpgradeable
```

Consequently, `DEFAULT_ADMIN_ROLE` remains its own role administrator and an existing administrator can immediately execute `grantRole(DEFAULT_ADMIN_ROLE, candidate)`. This creates an additional root administrator without a pending-transfer state, acceptance step, or delay. The candidate can subsequently revoke the original administrator's role.

The impact is amplified by the custom `hasRole` implementation, which treats every default administrator as holding every role. A newly granted administrator therefore immediately gains minting, burning, pausing, freezing, forced-transfer, metadata-management, and other module privileges. In `CMTATUpgradeableUUPS`, the same behavior also makes the new administrator satisfy `onlyRole(PROXY_UPGRADE_ROLE)` without an explicit upgrade-role grant. A compromised or erroneous administrator transaction can therefore result in an immediate and complete administrative handover that does not have the safeguards described by the protocol's trust model.

11 of 24
Findings

contracts/modules/5_CMTATBaseERC20CrossChain.sol

contracts/modules/0_CMTATBaseCommon.sol

contracts/modules/3_CMTATBaseRuleEngine.sol

contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol

Reentrant ruleEngine hook lets an approved spender drain frozen tokens via nested transferFrom calls

• Info

`contracts/modules/5_CMTATBaseERC20CrossChain.sol:29-43` delegates `transferFrom` into `CMTATBaseCommon.transferFrom`, which performs the transfer validation hook before spending allowance or moving balances:

```
function transferFrom(address from, address to, uint256 value) public returns (bool) {  
    _checkTransferred(_msgSender(), from, to, value);  
    return ERC20BaseModule.transferFrom(from, to, value);  
}
```

In this inheritance chain, `_checkTransferred` resolves to `CMTATBaseRuleEngine._checkTransferred`, which first checks active balance and then calls `ValidationModuleRuleEngine._transferred`. `_transferred` makes an external call to `ruleEngine.transferred(...)` before `_spendAllowance` and before `ERC20Upgradeable._transfer` execute.

That ordering allows a malicious or compromised configured rule engine to reenter `transferFrom` recursively while every frame still observes the same pre-transfer `balanceOf(from)` and `frozenTokens(from)` state. Each nested frame therefore passes `_checkActiveBalanceAndRevert(from, value)` using the sender's initial active balance. When the recursion unwinds, every frame then independently executes `_spendAllowance` and `_transfer`. Because `_transfer` only checks total balance, not the post-reentrancy active balance, the cumulative amount transferred can exceed the sender's initial active balance and consume frozen tokens.

Concrete example: if `from` has balance `100`, frozen balance `80`, active balance `20`, and has approved an attacker-controlled spender contract for `100`, a malicious rule engine can stack five nested `transferFrom(from, attacker, 20)` calls before any state changes occur. Each call passes the active-balance check against the original state. On unwind, the five frames transfer a total of `100`, draining `80` frozen tokens even though only `20` were unfrozen. This breaks the module's freeze invariant and allows theft of escrowed / compliance-locked balances through the standard delegated-transfer path.

✦ 12 of 24 Findings

contracts/modules/wrapper/options/DocumentEngineModule.sol

setDocument emits raw user inputs instead of verifying and reflecting the actual engine outcome[Info](#)

In `DocumentEngineModule.setDocument`, the function forwards the input `name`, `uri`, and `documentHash` to the external `documentEngine` and immediately emits a `DocumentUpdated` event using those raw `calldata` inputs. Unlike `removeDocument`, which safely queries the engine for the pre-state to emit an accurate removal event, `setDocument` fails to query the post-state.

If the external document engine normalizes the data (e.g., standardizing a URI format, stripping whitespace), enforces input constraints, or silently ignores the update without reverting (e.g., due to an active write-lock or internal access control failure in the engine), the token's emitted event will incorrectly broadcast the user's initial intent rather than the actual state stored in the engine. Because off-chain indexers and standard ERC-1643 observers rely entirely on these events, they will cache incorrect document attributes that permanently diverge from what `getDocument()` actually returns, breaking the standard's core invariant.

✦ 13 of 24 Findings

contracts/modules/wrapper/options/DocumentEngineModule.sol

Document engine setter accepts self and arbitrary non-compliant addresses, breaking ERC-1643 reads and delegated document management

• Info

`setDocumentEngine()` accepts any address except the current one, and all ERC-1643 operations then trust that stored target:

```
`function setDocumentEngine(IERC1643 documentEngine_) public ... {
    DocumentEngineModuleStorage storage $ = getDocumentEngineModuleStorage();
    require(address($._documentEngine) != address(documentEngine_),
        CMTAT_DocumentEngineModule_SameValue());
    setDocumentEngine($, documentEngine);
}

function getDocument(bytes32 name) public view ... returns (string memory uri, bytes32 documentHash, uint256
    lastModified) {
    DocumentEngineModuleStorage storage $ = _getDocumentEngineModuleStorage();
    if (address($._documentEngine) != address(0)) {
        return $_documentEngine.getDocument(name);
    } else {
        return ("", 0x0, 0);
    }
}

function getAllDocuments() public view ... returns (bytes32[] memory documentNames_) {
    DocumentEngineModuleStorage storage $ = getDocumentEngineModuleStorage();
    if (address($._documentEngine) != address(0)) {
        documentNames = $_documentEngine.getAllDocuments();
    }
}

function setDocument(bytes32 name, string calldata uri, bytes32 documentHash) public ... {
    require(address($._documentEngine) != address(0), CMTAT_DocumentEngineModule_NoDocumentEngine());
    $_documentEngine.setDocument(name, uri, documentHash);
    emit DocumentUpdated(name, uri, documentHash);
}`
```

There is no validation that the configured engine has code, is not `address(this)`, or actually implements `IERC1643`. If a concrete deployment exposes these inherited functions and the engine is set to `address(this)`, `getDocument()` and `getAllDocuments()` recurse back into the token until out-of-gas; `removeDocument()` also recurses because it first calls `getDocument()` on the configured engine. `setDocument()` re-enters the token through an external self-call and, unless the token has explicitly authorized itself as a document manager, reverts under the nested access-control check, so updates become unavailable.

If the engine is an EOA or another non-compliant target, delegated reads revert during ABI decoding, and `removeDocument()` reverts when its preliminary `getDocument()` call cannot decode a valid response. For `setDocument()`, the forwarded call has no return value, so an EOA target can accept it as a no-op and the token

still emits `DocumentUpdated` even though no document state was actually persisted anywhere. This leaves the ERC-1643 surface broken or inconsistent until the engine is reconfigured.

✦ 14 of 24 Findings

contracts/modules/wrapper/options/DebtEngineModule.sol

Debt engine setter accepts self and arbitrary non-compliant addresses, which can brick debt metadata reads

• Info

`setDebtEngine()` stores any address except the current one, and the read path blindly delegates to that stored address whenever it is non-zero:

```
`function setDebtEngine(IDebtEngine debtEngine_) public ... {
    DebtModuleStorage storage $ = getDebtEngineModuleStorage();
    require(address($_debtEngine) != address(debtEngine), CMTAT_DebtEngineModule_SameValue());
    setDebtEngine($, debtEngine);
}
```

```
function creditEvents() public view ... returns (CreditEvents memory creditEvents_) {
    DebtModuleStorage storage $ = getDebtEngineModuleStorage();
    if (address($_debtEngine) != address(0)) {
        creditEvents = $_debtEngine.creditEvents();
    }
}
```

```
function debt() public view ... returns (DebtInformation memory debtInformation_) {
    DebtModuleStorage storage $ = getDebtEngineModuleStorage();
    if (address($_debtEngine) != address(0)) {
        debtInformation = $_debtEngine.debt();
    }
}
```

There is no validation that `debtEngine_` has code, is different from `address(this)`, or actually behaves like `IDebtEngine`. In a concrete deployment that exposes these inherited getters, setting the engine to `address(this)` makes `debt()` and `creditEvents()` perform an external call back into the same contract and re-enter the same getter again, leading to unbounded recursion until the call runs out of gas. Setting the engine to an EOA or incompatible contract makes the delegated call return empty or malformed returndata, which then reverts during ABI decoding. As a result, one bad privileged configuration can disable debt metadata reads for all callers until the engine is changed again.

contracts/modules/internal/ERC20EnforcementModuleInternal.sol

15 of 24 Findings

contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol

contracts/modules/0_CMTATBaseCommon.sol

setFrozenTokens can freeze the zero address and brick all mint-based issuance paths

[Info](#)

`setFrozenTokens()` is wired to `ERC20EnforcementModuleInternal._setFrozenTokens()`, and that helper overwrites `_frozenTokens[account]` without rejecting `account == address(0)`. This differs from `_freezePartialTokens()` / `_unfreezePartialTokens()`, which explicitly reject the zero address.

Once `getFrozenTokens(address(0))` is set to any non-zero value, the common mint validation path starts failing for every non-zero issuance. `CMTATBaseCommon._mintOverride()` validates mints by calling `_checkTransferred(_msgSender(), address(0), account, value)`, and `_checkTransferred()` delegates to `ERC20EnforcementModuleInternal._checkActiveBalanceAndRevert(from, value)`. For `from == address(0)`, `balanceOf(address(0))` is zero, so the `_checkActiveBalance` branch `if (frozenTokensLocal >= balance)` is always true whenever the zero address has a non-zero frozen amount; any mint with `value > 0` therefore returns invalid and reverts.

This lets an `ERC20ENFORCER_ROLE` holder use a function that is supposed to freeze holder balances to instead create a protocol-wide issuance denial of service. Ordinary `mint`, `batchMint`, `crosschainMint`, and the mint leg of `burnAndMint` all become unusable until the zero-address freeze is cleared. On bridged deployments, destination-side `crosschainMint` failures can also interrupt cross-chain settlement and leave user transfers pending or stranded until an administrator intervenes.

contracts/modules/8_CMTATBaseHolderList.sol

16 of 24 Findings

contracts/modules/0_CMTATBaseCommon.sol

contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol

Reentrant rule-engine hook lets an approved spender bypass frozen-balance checks and drain frozen tokens

[Info](#)

`CMTATBaseHolderList.transfer()` is only a wrapper, but it preserves an unsafe execution order from the inherited standard path: the sender's active-balance check is performed, then an external rule-engine hook is called, and only afterwards are balances moved.

```
function transfer(address to, uint256 value) public virtual override(ERC20Upgradeable)
returns (bool) {
    address from = _msgSender();
    _checkTransferred(address(0), from, to, value);
    ERC20Upgradeable._transfer(from, to, value);
    return true;
}
```

In the rule-engine variant, `_checkTransferred()` reaches an external call before `_transfer` executes:

```
function _transferred(address spender, address from, address to, uint256 value) internal
virtual {
    _canTransferGenericByModuleAndRevert(spender, from, to);
    IRuleEngine ruleEngine_ = ruleEngine();
    if (address(ruleEngine_) != address(0)) {
        if (spender != address(0)) {
            ruleEngine_.transferred(spender, from, to, value);
        } else {
            ruleEngine_.transferred(from, to, value);
        }
    }
}
```

Because the hook is external and there is no reentrancy guard or post-hook revalidation, a reentrant path can consume the sender's remaining active balance after the outer call has already passed

`_checkActiveBalanceAndRevert`, but before the outer `ERC20Upgradeable._transfer` runs. The final ERC-20 transfer then checks only total balance, not active balance, so it can move tokens that should still be frozen.

A concrete exploit is: a victim has `balance=100`, `frozen=60`, `active=40`, and has approved an attacker-controlled spender for at least `80`. The attacker starts `transferFrom(victim, attacker, 40)`. The outer call passes the active-balance check and enters `ruleEngine_.transferred(...)`. During that callback, the attacker reenters `transferFrom(victim, attacker, 40)` through the approved spender. The inner call consumes the 40 active tokens, leaving `balance=60`, `frozen=60`, `active=0`. When control returns, the outer call still proceeds to `ERC20Upgradeable._transfer(victim, attacker, 40)` because the victim's raw ERC-20 balance is still 60. Net result: 80 tokens are drained even though only 40 were transferable, and 40 frozen tokens are moved in violation of the freeze model.

This breaks the HolderList variant's guarantee that standard transfers can only move unfrozen balance. Any reentrant rule-engine implementation, or any rule engine that can be induced to call attacker-controlled code during `transferred()`, can turn ordinary approved spenders into a frozen-token theft primitive.

17 of 24
Findings

contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol
contracts/modules/internal/ERC20EnforcementModuleInternal.sol
contracts/modules/0_CMTATBaseCommon.sol
contracts/modules/wrapper/options/ERC20CrossChainModule.sol

Missing zero-address validation in ERC20EnforcementModule.setFrozenTokens can block all non-zero mint paths

• Info

The root cause is that the ERC20 enforcement setter accepts `address(0)` and stores an arbitrary frozen-token amount for it, while every common mint path validates mints as if tokens were moving from `address(0)`. Once `frozenTokens[address(0)]` becomes non-zero, `_checkActiveBalance(address(0), value)` sees `balanceOf(address(0)) == 0` and rejects every non-zero mint before state mutation. An address with `ERC20ENFORCER_ROLE` can therefore call `setFrozenTokens(address(0), 1)` and immediately brick ordinary `mint`, `batchMint`, `crosschainMint`, and the mint leg of `burnAndMint`; the outage persists until another authorized actor explicitly resets the zero-address frozen amount.

```
function setFrozenTokens(address account, uint256 value)
public virtual override(IERC7943FungibleEnforcementSpecific) onlyERC20Enforcer
returns (bool result) {
    return _setFrozenTokens(account, value);
}

function _setFrozenTokens(address account, uint256 value) internal virtual returns(bool) {
    ERC20EnforcementModuleStorage storage $ = _getEnforcementModuleStorage();
    uint256 frozenTokensLocal = $_frozenTokens[account];
    ...
    $_frozenTokens[account] = value;
    ...
}
```

```
function _mintOverride(address account, uint256 value) internal virtual
override(ERC20MintModuleInternal) {
    _checkTransferred(_msgSender(), address(0), account, value);
    ERC20MintModuleInternal._mintOverride(account, value);
}

function _checkActiveBalance(address from, uint256 value) internal virtual view returns(bool
isValid, uint256 activeBalance){
    uint256 frozenTokensLocal = _getFrozenTokens(from);
    if(frozenTokensLocal > 0 ){
        uint256 balance = ERC20Upgradeable.balanceOf(from);
        if (frozenTokensLocal >= balance) {
            if (value == 0) {
                return (true, 0);
            }
        }
        return (false, 0);
    }
}
```

The impact is protocol-wide issuance denial of service on any deployment that inherits the common mint path: primary issuance stops, inbound bridge minting stops, and workflows that rely on burn-then-remint fail on the mint leg.

18 of 24 Findings

- contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol
- contracts/modules/0_CMTATBaseCommon.sol
- contracts/modules/3_CMTATBaseRuleEngine.sol

External call to RuleEngine before state mutations violates Checks-Effects-Interactions, enabling Read-Only Reentrancy

[Info](#)

The token implementation calls the external `ruleEngine_.transferred` hook during `_checkTransferred` before any state mutations (such as updating `_balances`, `totalSupply`, or `allowance`) are applied. By placing this state-mutating pre-transfer hook before the actual ERC-20 writes, the token contract violates the Checks-Effects-Interactions pattern.

If the configured Rule Engine makes any external calls or callbacks during its execution (or if the Rule Engine is compromised and manipulated to do so), an attacker can exploit this state discrepancy. Any DeFi protocol, vault, or voting mechanism that queries the token's `balanceOf` or `totalSupply` during the Rule Engine's execution will receive the stale pre-transfer state. An attacker can initiate a large transfer or burn, trigger the Rule Engine hook, and reenter a target protocol to manipulate collateral valuation, share pricing, or voting power before the token's balances are updated. While internal token state double-spending is prevented because the ERC-20 operations (`_transfer`, `_spendAllowance`) query fresh state and revert if insufficient, the external ecosystem remains vulnerable to the stale state exposure.

19 of 24
Findings

contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol
contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleERC1404.sol

`setRuleEngine()` accepts incompatible rule engines without validating the interfaces the rest of the module assumes

• Info

The rule-engine configuration path accepts incompatible engines, so `setRuleEngine()` can store an address that later breaks compliance checks or silently bypasses them.

`ValidationModuleERC1404` explicitly casts the configured engine to `IRuleEngineERC1404` inside `detectTransferRestriction`, `detectTransferRestrictionFrom`, and `messageForTransferRestriction`. If an administrator sets a valid ERC-3643 engine that does not implement the extended `IRuleEngineERC1404` methods, downstream calls to the ERC-1404 interface revert. This completely bricks the ERC-1404 prediction and messaging layer for all external wallets and integrators attempting to read compliance status prior to execution.

If the rule engine is mistakenly set to an Externally Owned Account (EOA), or to a contract lacking the expected interface, the protocol exhibits a dangerous silent failure. During token transfers, the `_transferred` hook calls `ruleEngine_.transferred(...)`. Because this function has no return value, the EVM low-level call to an EOA succeeds silently without executing any compliance logic. Consequently, transfers will proceed normally, completely circumventing all intended compliance rules without any transaction revert or indication of failure. Conversely, external protocols such as DEXes that call the `canTransfer` or `canTransferFrom` view functions will experience reverts, because these functions call `ruleEngine_.canTransfer(...)`, which expects a `bool` return value and the empty return data from the EOA causes the ABI decoder to revert.

 20 of 24
Findingscontracts/modules/wrapper/options/ERC20CrossChainModule.sol
contracts/modules/5_CMTATBaseERC20CrossChain.sol**Pausing disables privileged burn interfaces that are documented to remain available**[Info](#)

The documentation states that pausing should block standard transfers but not privileged burn operations. In the ERC20CrossChain variant, the exposed burn interfaces are wired to authorization hooks that additionally require the token to be unpaused:

```
function burnFrom(address account, uint256 value)
    public virtual override(IBurnFromERC20) onlyBurnerFrom
{
    address sender = _msgSender();
    _burnFrom(sender, account, value);
}

function burn(uint256 value) public virtual override(IBurnFromERC20) onlySelfBurn {
    address sender = _msgSender();
    _burnFromOperator(sender, sender, value);
}

function _authorizeBurnFrom() internal virtual override(ERC20CrossChainModule)
    onlyRole(BURNER_FROM_ROLE) whenNotPaused {}

function _authorizeSelfBurn() internal virtual override(ERC20CrossChainModule)
    onlyRole(BURNER_SELF_ROLE) whenNotPaused {}
```

This means `burnFrom` and `burn` revert during pause even for callers holding the required burner roles. The contradiction is security-relevant because the pause mechanism no longer behaves as the documented emergency mode where issuer-controlled supply reductions can still proceed, which can block redemptions, bridge-pool burns, or other privileged supply-management actions while the system is paused.

21 of 24
Findings contracts/modules/6_CMTATBaseERC2612.sol
contracts/modules/wrapper/core/TokenAttributeModule.sol**Mutable token name desynchronizes the EIP-712 domain separator, breaking ERC-2612 `permit` signatures**[Info](#)

The protocol supports ERC-2612 permits for gasless approvals and allows an authorized admin to modify the token's name via `TokenAttributeModule.setName()`.

During contract initialization, `CMTATBaseERC2612.__CMTAT_openzeppelin_init_unchained` sets up the EIP-712 domain separator by passing the initial token name to OpenZeppelin's `__EIP712_init_unchained`. OpenZeppelin's implementation hashes this name and caches it immutably to construct the EIP-712 domain separator. It does not provide a mechanism to update this cached name hash after initialization.

When an admin subsequently changes the token name via `setName()`, the EIP-712 domain separator remains permanently tied to the original name. When users attempt to sign `permit` messages, modern wallets and standard dApps dynamically fetch the token's current `name()` to build the EIP-712 domain separator. The resulting signatures will be generated using the new name. However, when these signatures are submitted on-chain, `ERC20PermitUpgradeable.permit` validates them against the cached domain separator containing the old name.

Because the domain separators mismatch, the cryptographic signature recovery will fail, causing all newly generated `permit` calls to revert. This results in a permanent denial of service for the `permit` functionality after the token name is modified.

22 of 24
Findings contracts/modules/wrapper/options/ERC7551Module.sol
contracts/modules/wrapper/extensions/ExtraInformationModule.sol**ERC7551 `setTerms` overload silently overwrites and deletes the document name**[Info](#)

The `ExtraInformationModule` manages shared token terms storage, which includes a `name`, `uri`, and `documentHash`. When the terms are updated using the ERC-7551 compatible `setTerms(bytes32, string)` function in `ERC7551Module`, the function internally constructs an `IERC1643CMTAT.DocumentInfo` struct by passing an empty string (`""`) for the `name` field.

This constructed struct is then passed to the parent's `_setTerms` function, which directly overwrites the entire terms storage object. If a valid document name was previously set via the standard `ExtraInformationModule.setTerms` function, calling the ERC-7551 `setTerms` overload will overwrite and silently delete the existing `name`. This causes unintended data loss and corrupts the on-chain metadata representing the tokenization terms, breaking assumptions for off-chain legal and compliance references.

23 of 24
Findings contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleERC1404.sol**detectTransferRestrictionFrom reports SPENDER_FROZEN before checking deactivated/paused state, violating the required restriction-code ordering**

• Best Practices

The stated ERC-1404 ordering invariant requires that `detectTransferRestriction` and `detectTransferRestrictionFrom` first return `DEACTIVATED` when the contract is deactivated, then `PAUSED` when it is paused, and only afterwards report frozen-participant codes (`FROM_FROZEN` / `TO_FROZEN` / `SPENDER_FROZEN`) before consulting the external rule engine. In `ValidationModuleERC1404.detectTransferRestrictionFrom`, the spender freeze check is evaluated first, before any deactivation or pause check:

```
function detectTransferRestrictionFrom(address spender, address from, address to, uint256
value) public view returns (uint8 code) {
    IRuleEngineERC1404 ruleEngine_ = IRuleEngineERC1404(address(ruleEngine()));
    if (isFrozen(spender)) {
        return uint8(IERC1404Extend.REJECTED_CODE_BASE.TRANSFER_REJECTED_SPENDER_FROZEN);
    } else {
        uint8 codeReturn = _detectTransferRestriction(from, to, value); // this contains
        deactivated()/paused() checks
        ...
    }
}
```

The deactivation and pause checks live inside `_detectTransferRestriction`, which is only reached in the `else` branch. Consequently, when the contract is deactivated (which implies it is paused) or merely paused, and the `spender` is also frozen, the function returns `TRANSFER_REJECTED_SPENDER_FROZEN` (code 5) instead of `TRANSFER_REJECTED_DEACTIVATED` (code 1) or `TRANSFER_REJECTED_PAUSED` (code 2). The actual on-chain enforcement path (`_canTransferStandardByModuleAndRevert`) instead reverts on `_requireNotPaused()` before evaluating frozen participants, so the predicted restriction reason returned by `detectTransferRestrictionFrom` diverges from the reason enforcement would actually raise. Because both codes are non-zero, the transfer is still correctly predicted to be blocked, so this is a reporting/state-drift inconsistency rather than a bypass, but integrators (e.g. exchanges) relying on the returned code to reason about the token lifecycle state will receive a misleading restriction reason.

contracts/modules/wrapper/core/ERC20BaseModule.sol

24 of 24 Findings

contracts/modules/wrapper/options/ERC20CrossChainModule.sol

contracts/interfaces/technical/IERC20Allowance.sol

Unconditional Spend emission in ERC20BaseModule.transferFrom and ERC20CrossChainModule._burnFrom

• Best Practices

Root cause: the allowance-spending entrypoints emit IERC20Allowance.Spend even when the allowance did not decrease, which contradicts the interface documentation that says Spend is omitted for infinite approvals. In `ERC20BaseModule.transferFrom`, the event is emitted after a successful transfer without checking whether allowance was actually reduced. In `ERC20CrossChainModule._burnFrom`, the event is emitted immediately after `_spendAllowance`, again without suppressing the infinite-allowance case.

```
bool result = ERC20Upgradeable.transferFrom(from, to, value);
if (result) {
    emit Spend(from, _msgSender(), value);
}
```

```
ERC20Upgradeable._spendAllowance(account, sender, value );
emit IERC20Allowance.Spend(account, sender, value);
_burnFromOperator(sender, account, value);
```

Exploit path: a user grants a spender an infinite approval, then the spender repeatedly uses `transferFrom` or, where authorized, `burnFrom`. Each successful call produces a Spend log even though the approval remains infinite. Any downstream system that mirrors allowance state or spend quotas from Spend events, following the documented semantics, can be poisoned into believing allowance was consumed when it was not.

Impact: event-driven allowance accounting, monitoring, or compliance tooling can drift from on-chain truth, leading to false denials, false alerts, or incorrect remaining-approval calculations.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.] No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.